

Many somewhat different, somewhat independent trees are better than one highly correlated tree.

"The key aspect of Random Forest is building multiple diverse, less-correlated decision trees using random sampling of data and features, then aggregating their predictions to reduce variance and overfitting."

Random Forest is a supervised machine learning algorithm used for both classification and regression. It works by creating many decision trees on different random samples of the training data and then combining their predictions.

- **Classification:** Each tree votes for a class → the class with the majority vote is selected.
- **Regression:** Predictions from all trees are averaged.
- **Why "Random"?** Each tree gets a random sample of rows, and at each split it considers a random subset of features.
- **Why "Forest"?** Because it consists of many decision trees.
- **Main advantage:** Usually more accurate and less prone to overfitting than a single decision tree.
- **Main disadvantage:** More computationally expensive and less interpretable than a single decision tree.

Example: for predicting whether a customer will churn, 100 decision trees may make predictions. If 72 predict Churn and 28 predict No Churn, Random Forest predicts Churn.

In one line:

Random Forest = many randomized decision trees + combined predictions.

The most important aspect of Random Forest is:

It combines predictions from many diverse decision trees to reduce overfitting and improve generalization.

The key idea is randomness + aggregation.

- Each tree is trained on a random bootstrap sample of the training data.
- At each split, only a random subset of features is considered.
- Final prediction is majority voting for classification or averaging for regression.

This randomness makes the trees less correlated with each other, which is crucial. Combining many less-correlated trees produces a model that is generally more stable and robust than a single decision tree.

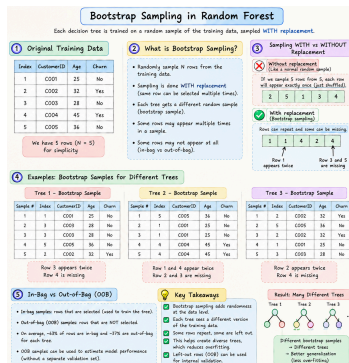
```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
tree = model.oob_prediction_[0]

# Create a large figure
plt.figure(figsize=(30, 15))

plot_tree(
    tree,
    feature_names=X.columns,
    filled=True,
    rounded=True,
    fontsize=8
)

# Save as high-resolution PNG
plt.savefig(
    "random_forest_tree.png",
    dpi=300,
    bbox_inches="tight"
)

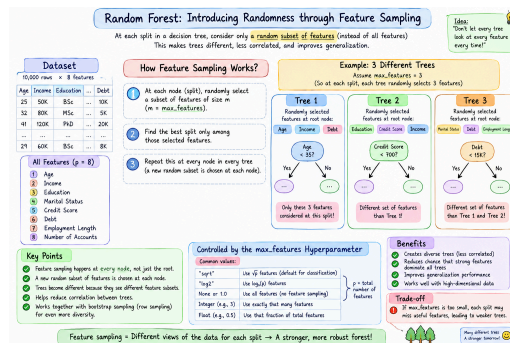
plt.show()
```



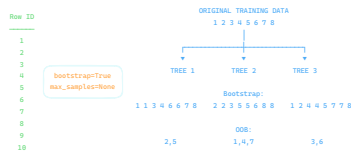
Original training data
n = 1,000 rows

Bootstrap sampling
WITH replacement

Tree 1 → 1,000 sampled rows
Tree 2 → 1,000 sampled rows
Tree 3 → 1,000 sampled rows



Original Dataset



In-Bag observations are training observations selected at least once in a tree's bootstrap sample and are used to train that tree. Out-of-Bag observations are those not selected for that particular tree. Since the tree has not trained on its OOB observations, they can be used to estimate its generalization performance. Random Forest aggregates these OOB predictions across trees to produce an overall OOB score.

Why approximately 63.2% In-Bag and 36.8% OOB?

Because observations can repeat, we don't normally get 10,000 unique observations. For any particular observation, the probability that it is not selected on one draw is:

$$1 - \frac{1}{N}$$

After m draws:

$$P(\text{never selected}) = \left(1 - \frac{1}{N}\right)^m$$

As m becomes large:

$$\left(1 - \frac{1}{N}\right)^m \approx e^{-1} \approx 0.368$$

Therefore:

$$P(\text{OOB}) \approx 36.8\%$$

and:

$$P(\text{selected at least once}) = 1 - 0.368 = 0.632$$

How Random Forest uses OOB observations

Suppose we have 100 trees ...

Consider observation #908

Tree 1 = In-Bag ❌ don't use for OOB prediction

Tree 2 = OOB ✅ predict

Tree 3 = OOB ✅ predict

Tree 4 = In-Bag ❌

Tree 5 = In-Bag ❌

Tree 6 = OOB ✅ predict

...

Tree 100 = OOB ✅ predict

Approximately 36.8% of trees will have that observation OOB.

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    bootstrap=True,
    max_samples=None,
    random_state=0
)

model.fit(X_train, y_train)

Then:

Python

model.oob_score_

For a classifier, the default OOB scoring is accuracy.

For example:

0.813

means approximately:

OOB Accuracy = 81.3%
```

After this:

```
model.oob_decision_function_
```

gives the OOB estimated (log) probabilities for each training observation.

For binary classification, you might see:

	Class 0	Class 1
Observation 1	0.10	0.90
Observation 2	0.70	0.30
Observation 3	0.10	0.90
Observation 4	0.90	0.10

These probabilities are calculated using only trees for which that observation was OOB.

For example:

Observation #1	Class 0 probabilities	Class 1 probabilities
Tree 6 (OOB) = {0.2, 0.8}	0.2	0.8
Tree 12 (OOB) = {0.4, 0.6}	0.4	0.6

This is useful if you want more than just the overall oob_score.

